



Test Summary Report

C06 - CityClean

Riferimento	
Versione	1.0
Data	14/12/2025
Destinatario	Prof.ssa Filomena Ferrucci Prof. Fabio Palomba
Presentato da	C06 Team Nu Spettacoli: ● Michele Bartoli (MB) ● Luigi Balsio (LB) ● Pio Cascone (PC) ● Domenico di Caprio (DC) ● Saverio Di Perna (SP) ● Antonio Pio Gargiulo (AG) ● Marco Ianniello (MI) ● Amanda Pia Caterina Robertazzi (AR) ● Vittorio Ruotolo (VR) ● Paolo Maria Siconolfi (PS) ● Davide Valiante (DV)
Approvato da	



Team Members

Nome	Ruolo	Acronimo	Contatti
Antonio Ferrentino	Project Manager	AF	a.ferrentino50@studenti.unisa.it
Francesco Perilli	Project Manager	FP	f.perilli2@studenti.unisa.it
Michele Bartoli	Team Member	MB	m.bartoli@studenti.unisa.it
Luigi Balsio	Team Member	LB	l.balsio@studenti.unisa.it
Pio Cascone	Team Member	PC	p.cascone6@studenti.unisa.it
Domenico di Caprio	Team Member	DC	d.dicaprio2@studenti.unisa.it
Saverio Di Perna	Team Member	SP	s.diperna@studenti.unisa.it
Antonio Pio Gargiulo	Team Member	AG	a.gargiulo65@studenti.unisa.it
Marco Ianniello	Team Member	MI	m.ianniello28@studenti.unisa.it
Amanda Pia Caterina Robertazzi	Team Member	AR	a.robertazzi5@studenti.unisa.it
Vittorio Ruotolo	Team Member	VR	v.ruotolo4@studenti.unisa.it
Paolo Maria Siconolfi	Team Member	PS	p.siconolfi1@studenti.unisa.it
Davide Valiante	Team Member	DV	d.valiante@studenti.unisa.it



Revision History

Data	Versione	Descrizione	Autori
13/12/2025	0.1	Stesura del documento	AG,PS
14/12/2025	1.0	Inserimento dei vari risultati di test	Tutto il Team



Sommario

[Team Members](#)

[Revision History](#)

[1. Introduzione](#)

[2. Relazione con altri documenti](#)

[3. Testing Unitario](#)

[4. Testing di Sistema](#)

[4.1 Metodologia e Strumenti \(Automazione con Maestro\)](#)

[4.2 Testing Manuale](#)

[4.3 Ambiente di Test](#)



1. Introduzione

Il sistema che si vuole realizzare ha come scopo principale quello di **incentivare i cittadini a una gestione più responsabile dei rifiuti**, in particolare quelli che tendono a finire dispersi nell'ambiente o in strada, attraverso un meccanismo di ricompensa basato sulla raccolta differenziata.

Attraverso la piattaforma, gli utenti saranno motivati a conferire specifiche tipologie di rifiuti (ad esempio plastica, lattine, mozziconi, ecc.) presso **punti di raccolta designati (ecopunti)** distribuiti sul territorio cittadino. Ogni conferimento darà diritto all'accumulo di punti o crediti, tracciati in tempo reale dall'applicazione.

All'interno del documento saranno presenti le strategie di testing che verranno adottate, le funzionalità testate e gli strumenti che saranno utilizzati per garantire una piattaforma con il minor numero possibile di errori e che funzioni in maniera efficace.

2. Relazione con altri documenti

Di seguito vengono elencate le relazioni tra il presente documento e gli altri documenti del testing:

- Relazione con il **Test Plan(TP)**:
 - Il Test Summary Report fa riferimento alle attività di testing specificate nel Test Plan
- Relazione con il **Test Case Specification(TCS)**:
 - Il Test Summary Report contiene una cernita dei test specificati nel Test Case Specification
- Relazione con il **Test Incident Report(TIR)**:
 - Il Test Summary Report contiene una cernita dei risultati dell'esecuzione specificati nel Test Incident Report

3. Testing Unitario

Dato l'utilizzo di **Supabase** come Backend-as-a-Service (BaaS), gran parte della logica di business risiede nella corretta orchestrazione tra il client Supabase e l'interfaccia utente. Per questo motivo, la strategia di testing si è evoluta spostando il focus dal testing unitario classico (con mocking dei dati manuale) verso una verifica funzionale più ampia.

Il testing delle singole unità di codice è stato effettuato manualmente in fase di sviluppo (developer testing) per garantire la correttezza logica dei metodi. Per la fase di validazione formale, invece di isolare i service con dei mock, abbiamo utilizzato **Maestro** per eseguire test di integrazione automatizzati. Questo approccio ci permette di verificare che le unità di



codice (Service e UI) cooperino correttamente con l'istanza reale di Supabase, intercettando errori che i soli test unitari con mock non avrebbero rilevato (es. errori reali di query, permessi RLS, o parsing dei dati JSON).

4. Testing di Sistema

L'attività di testing di sistema è stata condotta con l'obiettivo di validare il comportamento dell'applicazione **CityClean** nel suo complesso, verificando che i requisiti funzionali e non funzionali siano soddisfatti in un ambiente il più possibile simile a quello di produzione.

L'approccio adottato è di tipo **Black-Box**: i test interagiscono con l'interfaccia utente senza conoscere la struttura interna del codice, simulando il comportamento di un utente reale.

4.1 Metodologia e Strumenti (Automazione con Maestro)

La strategia principale si basa sull'automazione dei test End-to-End (E2E) utilizzando il framework **Maestro**. A differenza dei test unitari isolati, questa metodologia ci permette di verificare l'intero stack dell'applicazione: dall'interazione UI (Flutter) fino alla persistenza dei dati (Supabase).

I test sono definiti tramite **Flow** (file .yaml) che descrivono sequenze di azioni utente. Ogni flow copre uno specifico caso d'uso (es. "Login Utente", "Riscatto Premio", "Navigazione Mappa"). L'esecuzione di questi flow verifica:

1. **Navigazione:** Il corretto passaggio tra le schermate (es. dalla Home alla sezione Premi).
2. **Stato della UI:** La visibilità di elementi critici (bottoni, testi, messaggi di errore) identificati tramite Semantics e ID univoci.
3. **Logica Condizionale:** La reazione dell'app a stati diversi (es. verifica che il bottone "Riscatto" sia disabilitato se i punti sono insufficienti).
4. **Integrazione Backend:** La conferma che le operazioni sulla UI (es. click su "Conferma") generino le corrette chiamate verso Supabase (verificato indirettamente tramite il feedback visivo di successo, come Toast o aggiornamento del saldo punti).

4.2 Testing Manuale

Nonostante l'ampio uso dell'automazione, alcune funzionalità specifiche sono state testate manualmente su emulatore e dispositivi fisici.

Il testing manuale è stato necessario per:

- **Funzionalità Hardware:** Interazioni che coinvolgono la fotocamera (es. scansione QR Code o foto ai rifiuti) e la geolocalizzazione GPS dinamica.
- **User Experience (UX):** Verifica della fluidità delle animazioni e della reattività al tocco.



- **Gestione Errori di Rete:** Simulazione manuale di assenza di connettività per verificare i messaggi di errore "Offline" (casi marcati come *N/A* nei test automatici).

4.3 Ambiente di Test

I test di sistema sono stati eseguiti nel seguente ambiente:

- **Frontend:** Emulatore Android configurato con l'ultima build dell'applicazione Flutter.
- **Backend:** Istanza reale di **Supabase**. Per garantire la ripetibilità dei test senza "sporcare" i dati di produzione, sono stati utilizzati account utente di test (es. test2@test.com) con dati pre-configurati o resettati prima di ogni esecuzione del flow (tramite script di pulizia o setup manuale).

In conclusione, questa strategia mista (Automazione Maestro + Testing Manuale) ha permesso di coprire i principali flussi critici dell'applicazione, garantendo che le feature implementate siano robuste e pronte per il rilascio.

Di seguito i risultati dei test:

Esecuzione	Successi	Fallimenti	N/A
Esecuzione 1	5	0	1
Esecuzione 2	4	0	1
Esecuzione 3	2	0	2
Esecuzione 4	2	0	2
Esecuzione 5	1	0	3
Esecuzione 6	4	0	1
Esecuzione 7	3	0	1
Esecuzione 8	1	0	3
Esecuzione 9	6	0	0
Esecuzione 10	4	0	2
Esecuzione 11	10	0	6